EduGenie

Google Gemini Powered Learning Assistant

Final Project Screenshots, Coding Evidence & Explanations

This document combines the EduGenie application screenshots with representative coding screenshots based on the project implementation and reference document.

API key values are intentionally not displayed in the coding evidence.

Application Screenshot

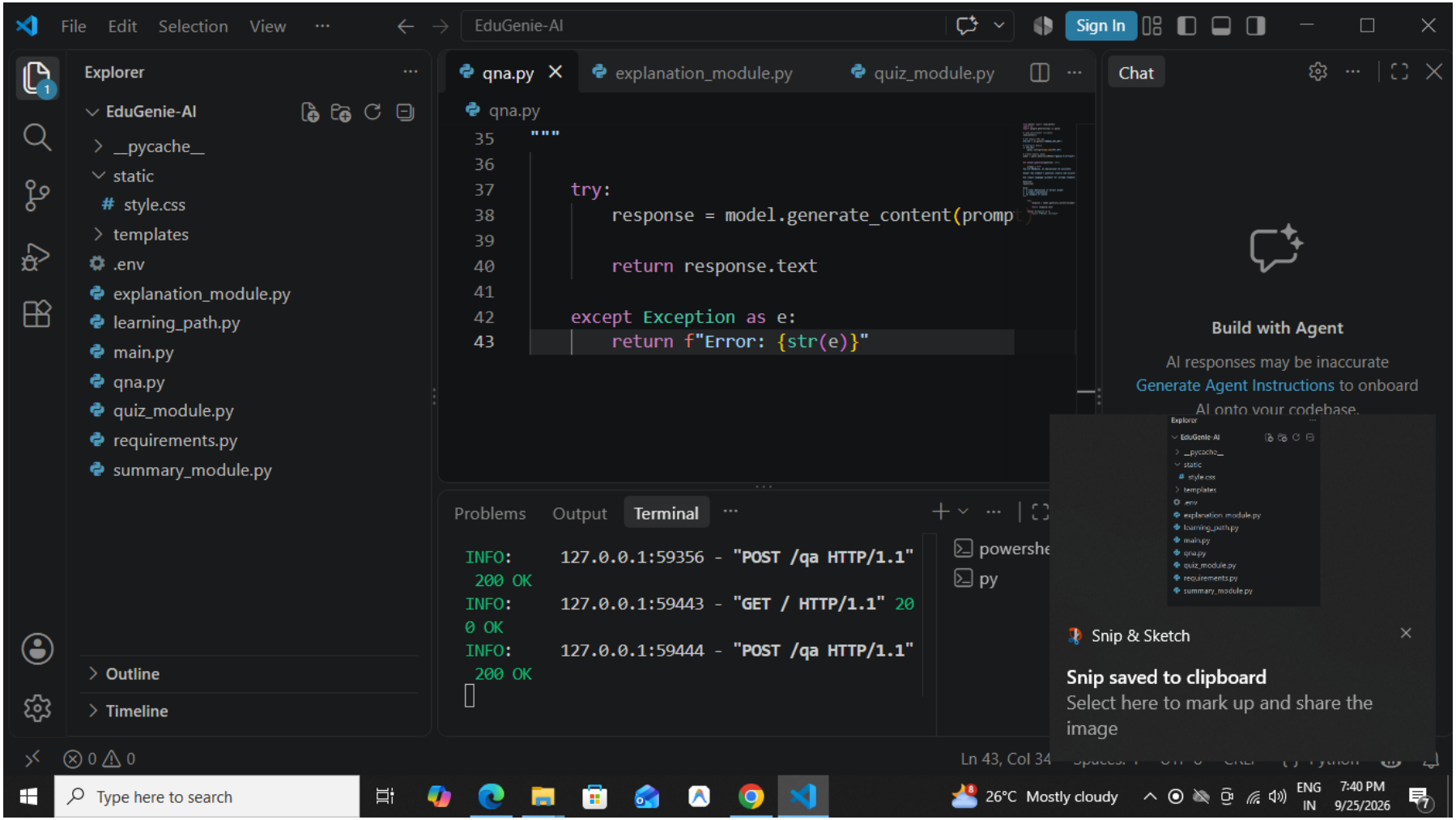


Fig. 1 – Backend Development and API Testing

Explanation: The backend is developed in Visual Studio Code using FastAPI. The terminal shows successful API requests, demonstrating that the application is running and responding.

Application Screenshot

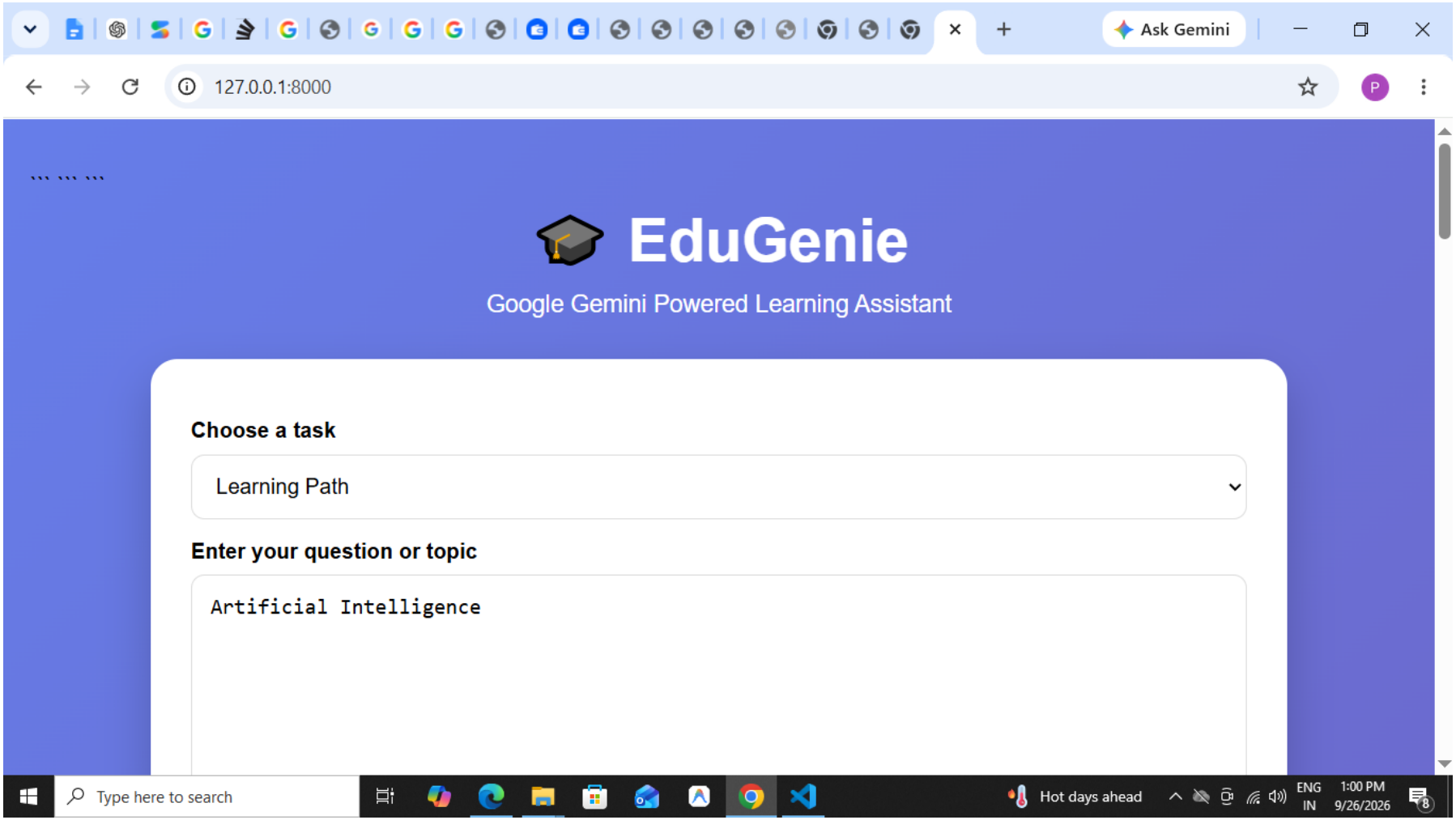


Fig. 2 – Learning Path Input Interface

Explanation: The user selects Learning Path and enters Artificial Intelligence as the topic. This provides a simple interface for requesting a personalized study plan.

Application Screenshot

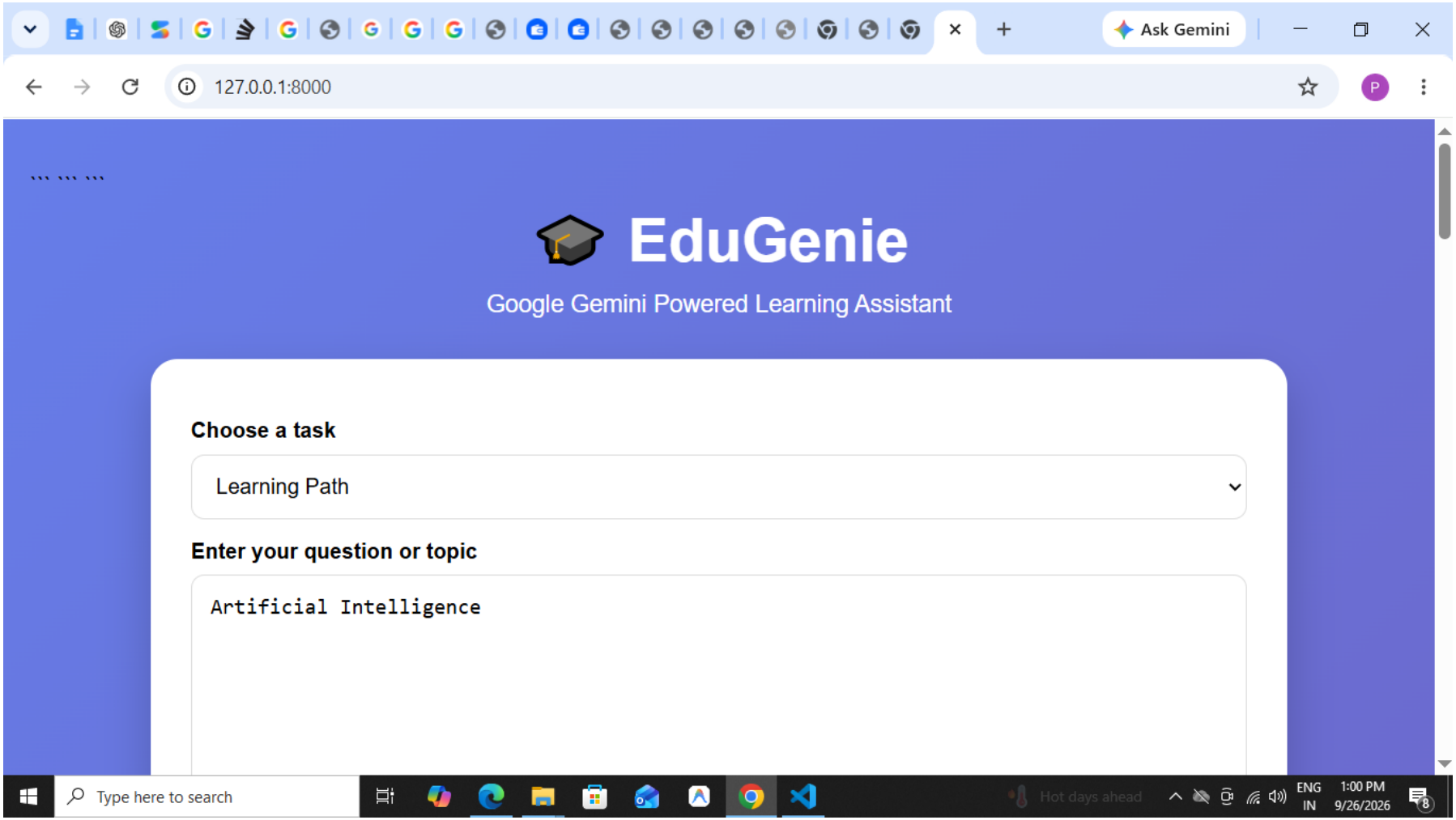


Fig. 3 – User Input and Task Selection

Explanation: The frontend lets students select an EduGenie feature and enter a topic or question before sending the request to the backend.

Application Screenshot

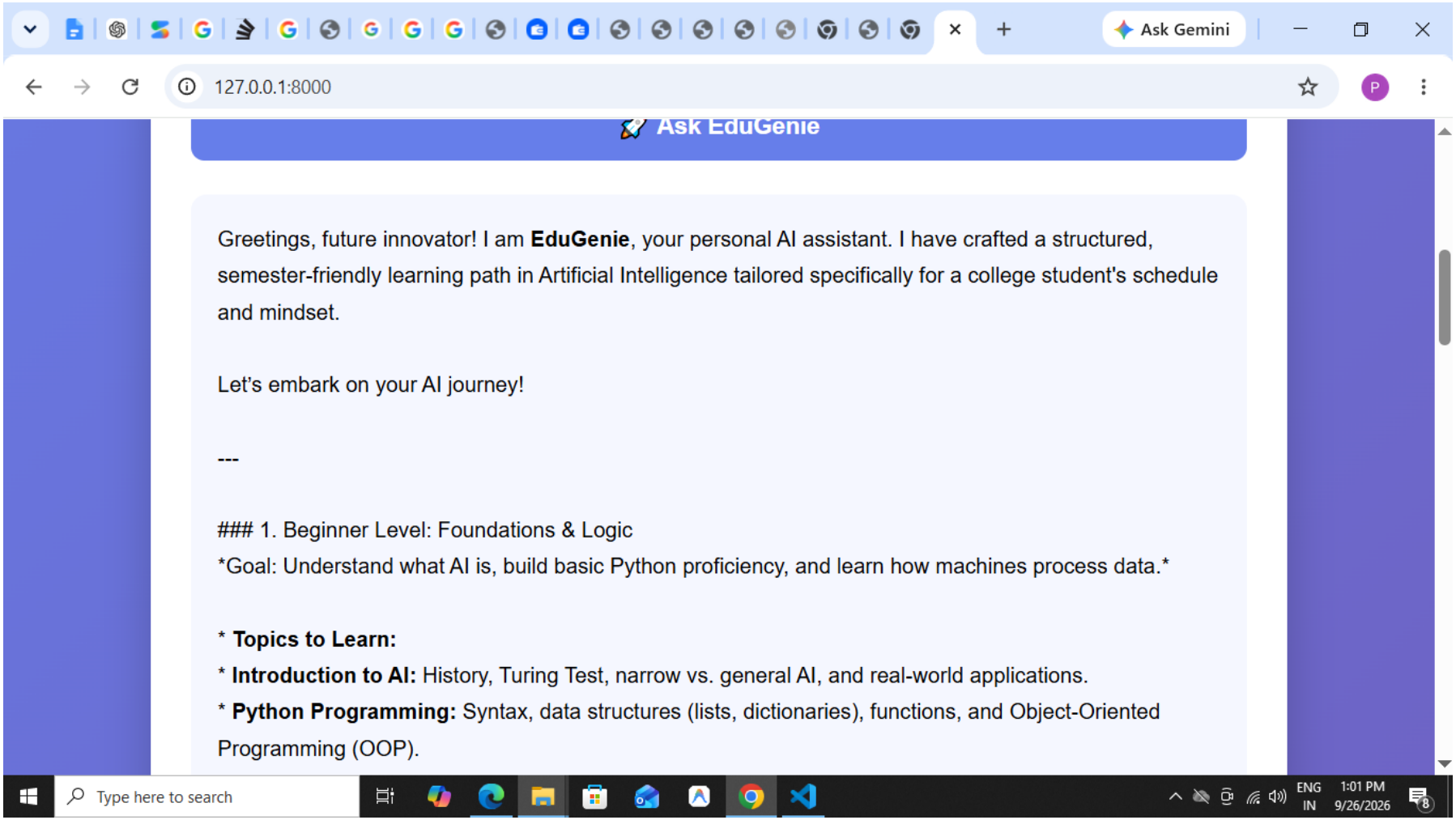


Fig. 4 – Generated Personalized Learning Path

Explanation: Google Gemini generates a structured beginner-to-advanced learning path with topics, explanations, activities, and resource suggestions.

Application Screenshot

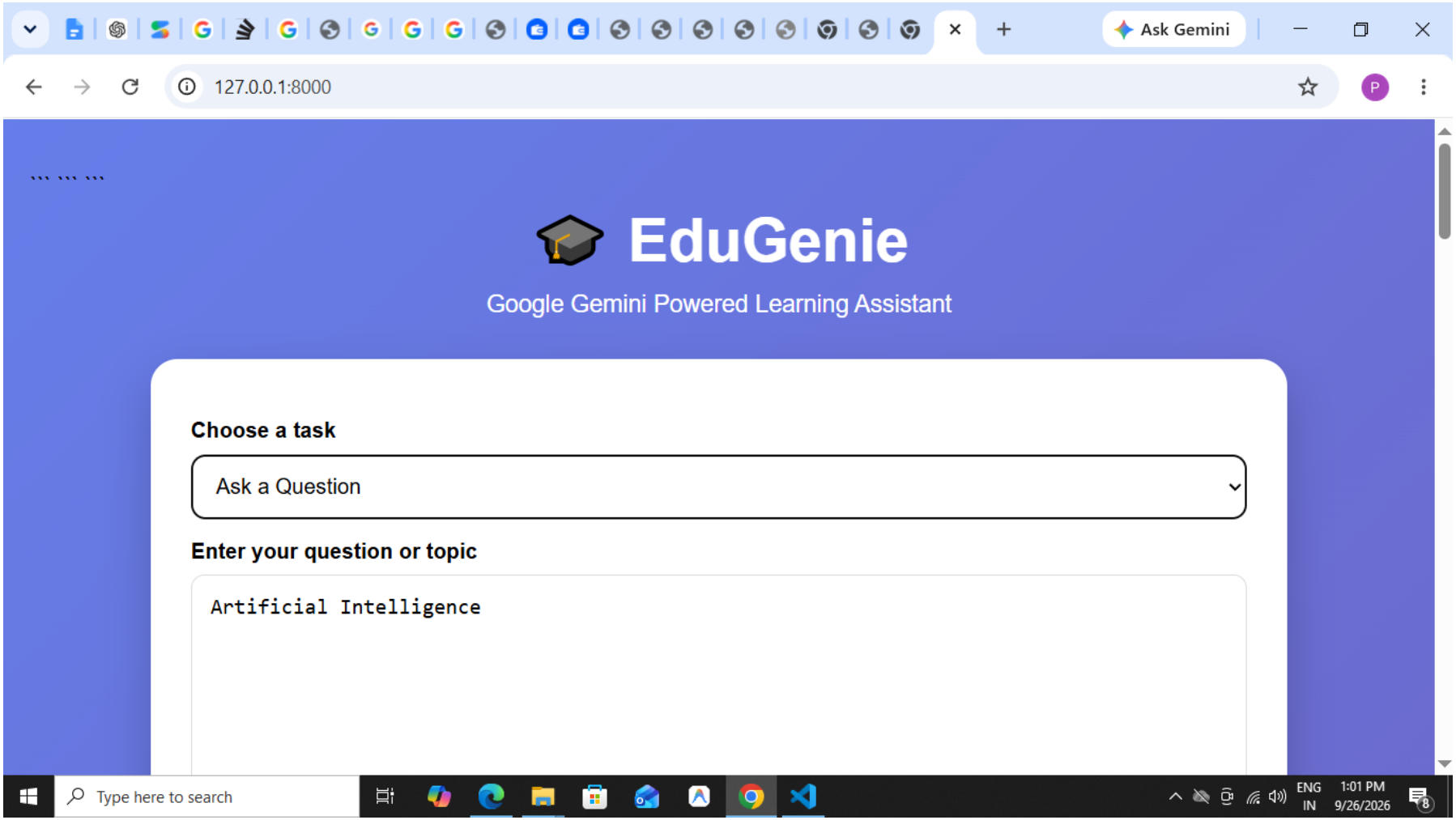


Fig. 5 – Ask a Question Feature

Explanation: The Ask a Question option sends the student's input to the Q&A; module and displays the generated educational response.

Application Screenshot

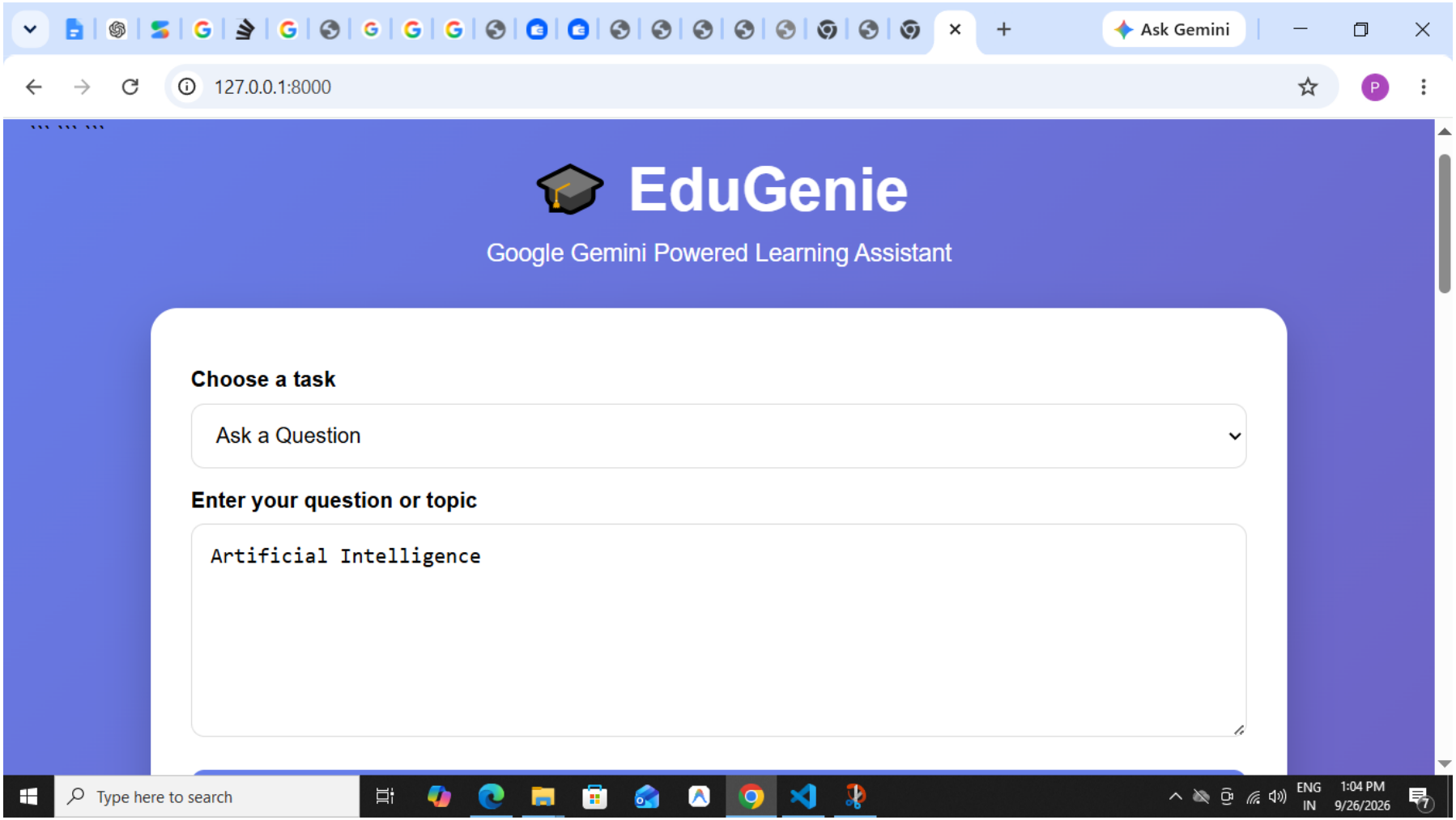


Fig. 6 – EduGenie Question and Answer Interface

Explanation: The responsive interface provides a common interaction area for the main EduGenie learning functions.

Coding Implementation

The following coding screenshots document the major modules used to implement EduGenie. Each module has a specific responsibility and is connected through the FastAPI backend.

Coding Screenshot

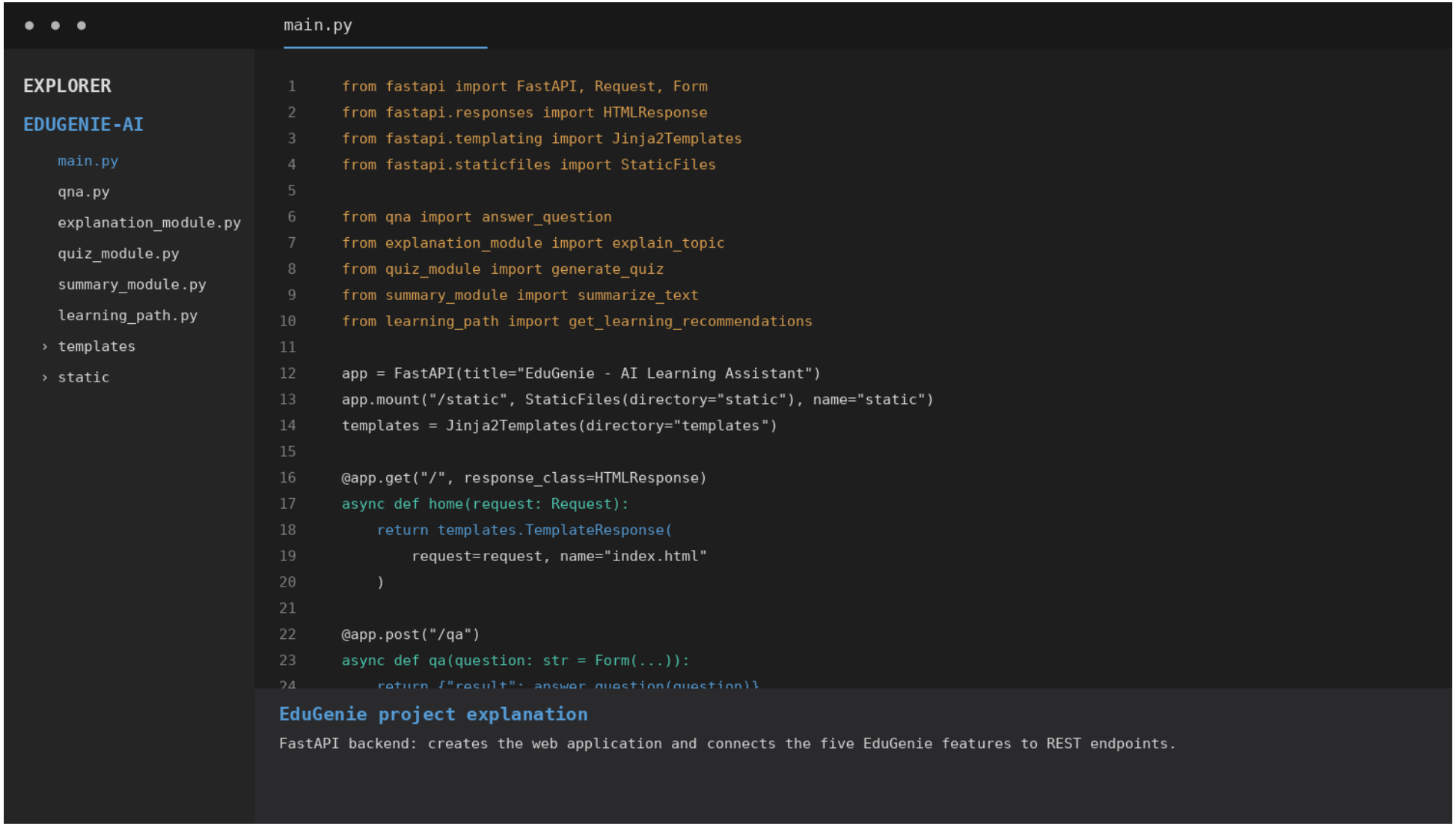


Fig. C1 – main.py

Explanation: FastAPI backend: creates the web application and connects the five EduGenie features to REST endpoints.

Coding Screenshot

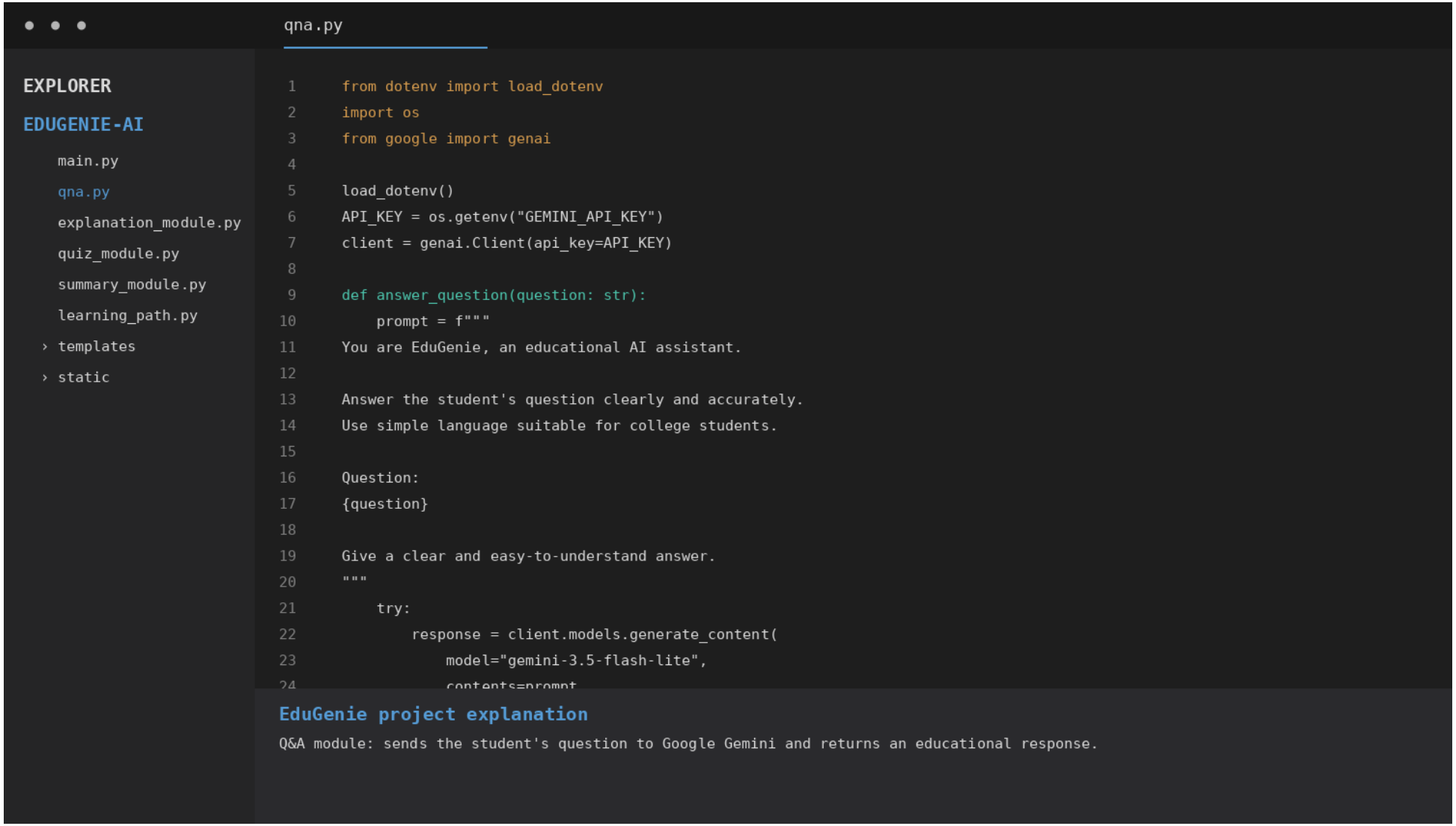


Fig. C2 – qna.py

Explanation: Q&A; module: sends the student's question to Google Gemini and returns an educational response.

Coding Screenshot

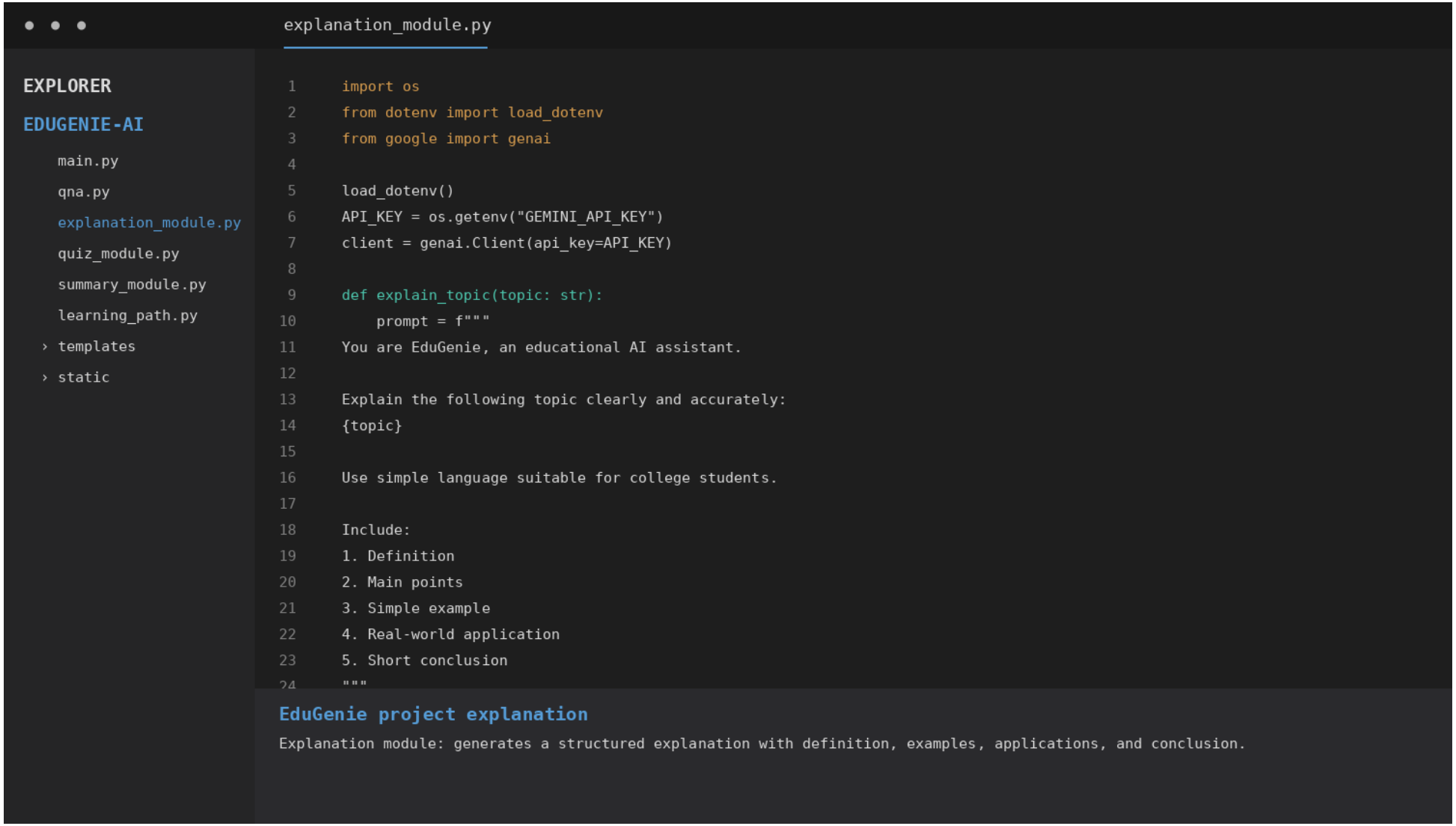


Fig. C3 – explanation_module.py

Explanation: Explanation module: generates a structured explanation with definition, examples, applications, and conclusion.

Coding Screenshot

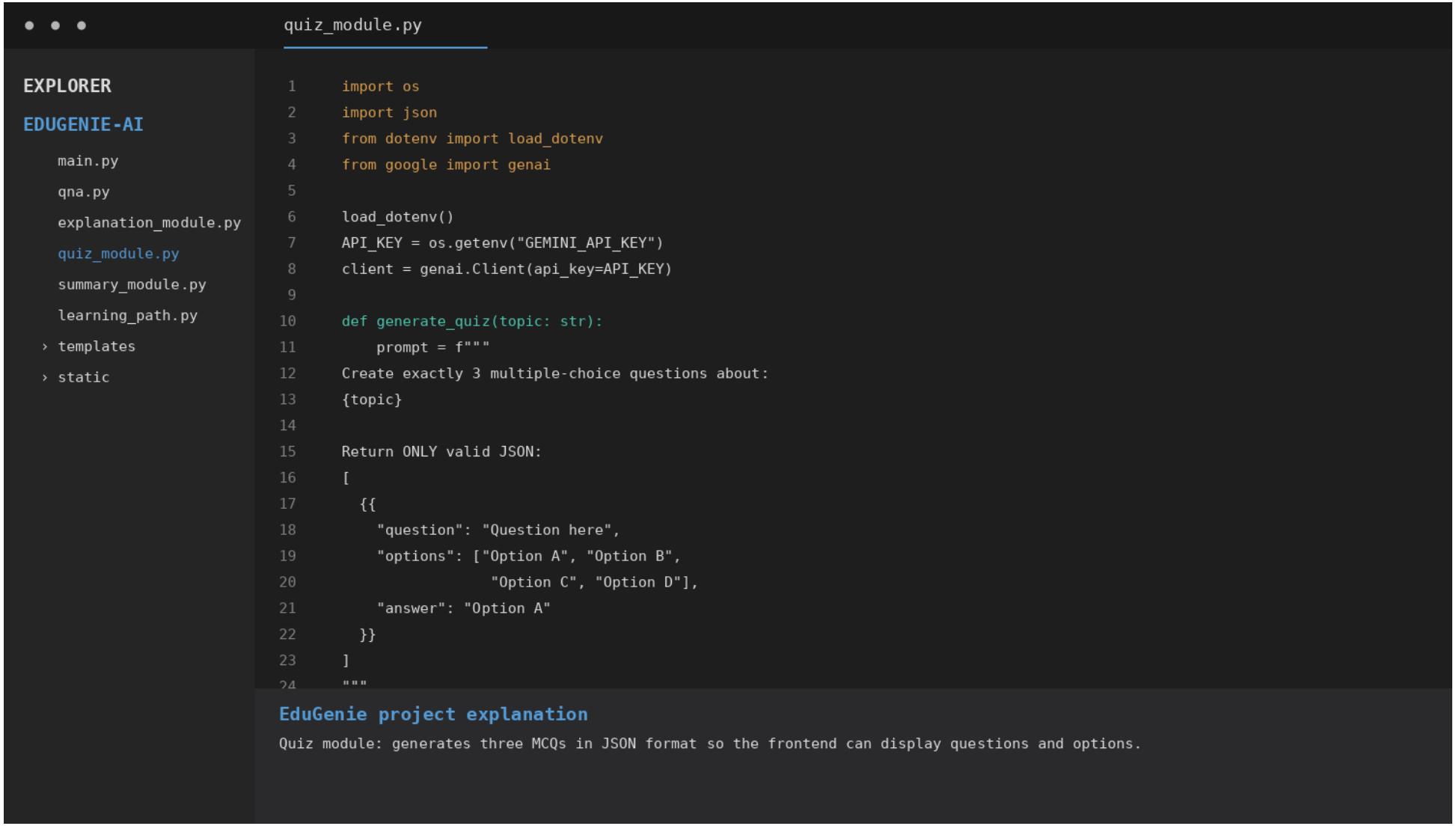


Fig. C4 – quiz_module.py

Explanation: Quiz module: generates three MCQs in JSON format so the frontend can display questions and options.

Coding Screenshot

summary_module.py

EXPLORER

EDUGENIE-AI

- main.py
- qna.py
- explanation_module.py
- quiz_module.py
- summary_module.py
- learning_path.py
- templates
- static

```python
import os
from dotenv import load_dotenv
from google import genai

load_dotenv()
API_KEY = os.getenv("GEMINI_API_KEY")
client = genai.Client(api_key=API_KEY)

def summarize_text(text: str):
    prompt = f"""
Summarize the following educational content.

Requirements:
- Keep the important points
- Use simple language
- Use bullet points where useful
- Do not remove important information

Content:
{text}
"""
    try:
        response = client.models.generate_content(
            model="gemini-3.5-flash-lite"
```

EduGenie project explanation

Summary module: converts long educational text into a concise, readable summary while preserving key points.

Fig. C5 – summary_module.py

Explanation: Summary module: converts long educational text into a concise, readable summary while preserving key points.

Coding Screenshot

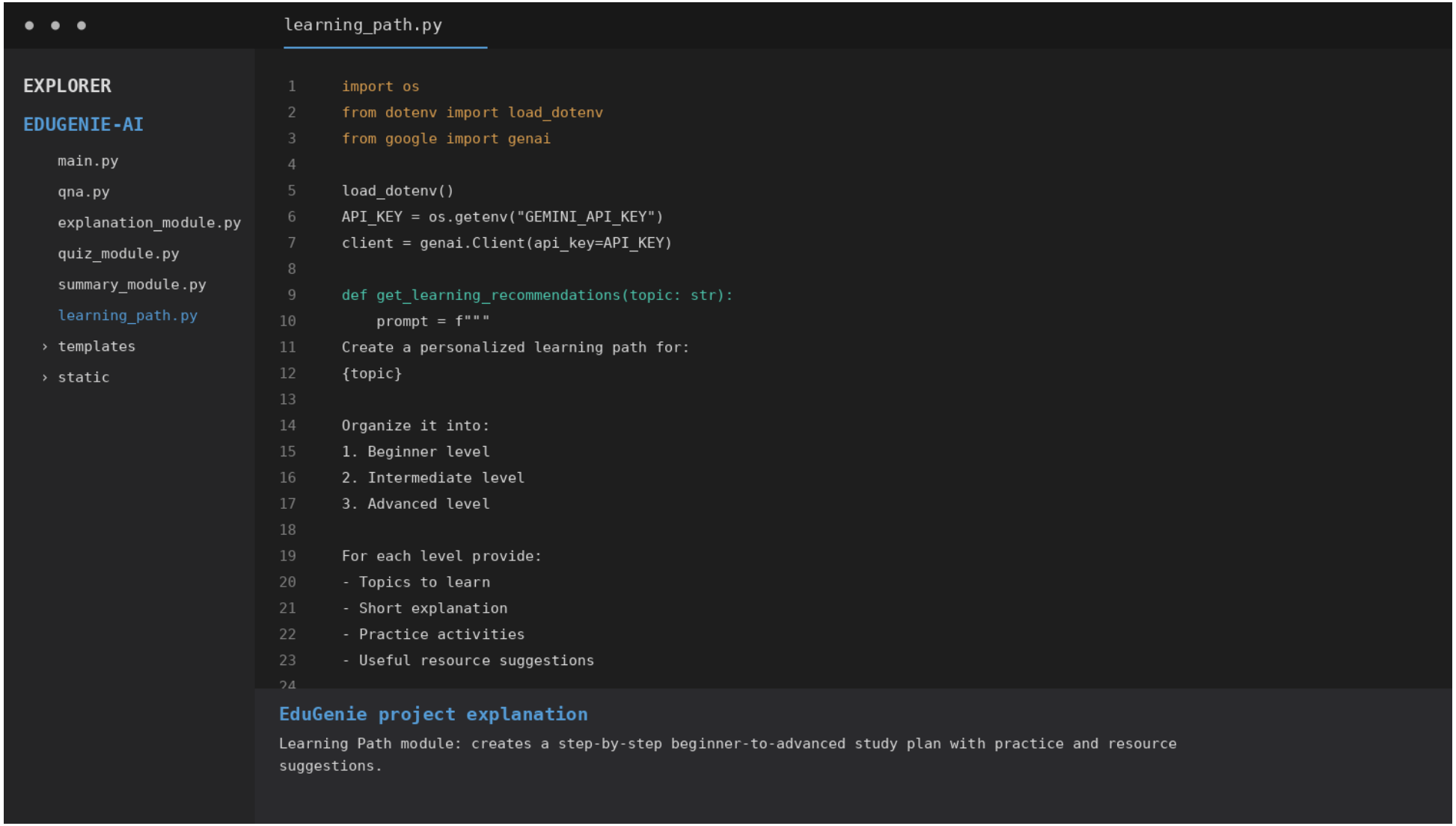


Fig. C6 – learning_path.py

Explanation: Learning Path module: creates a step-by-step beginner-to-advanced study plan with practice and resource suggestions.

Coding Screenshot

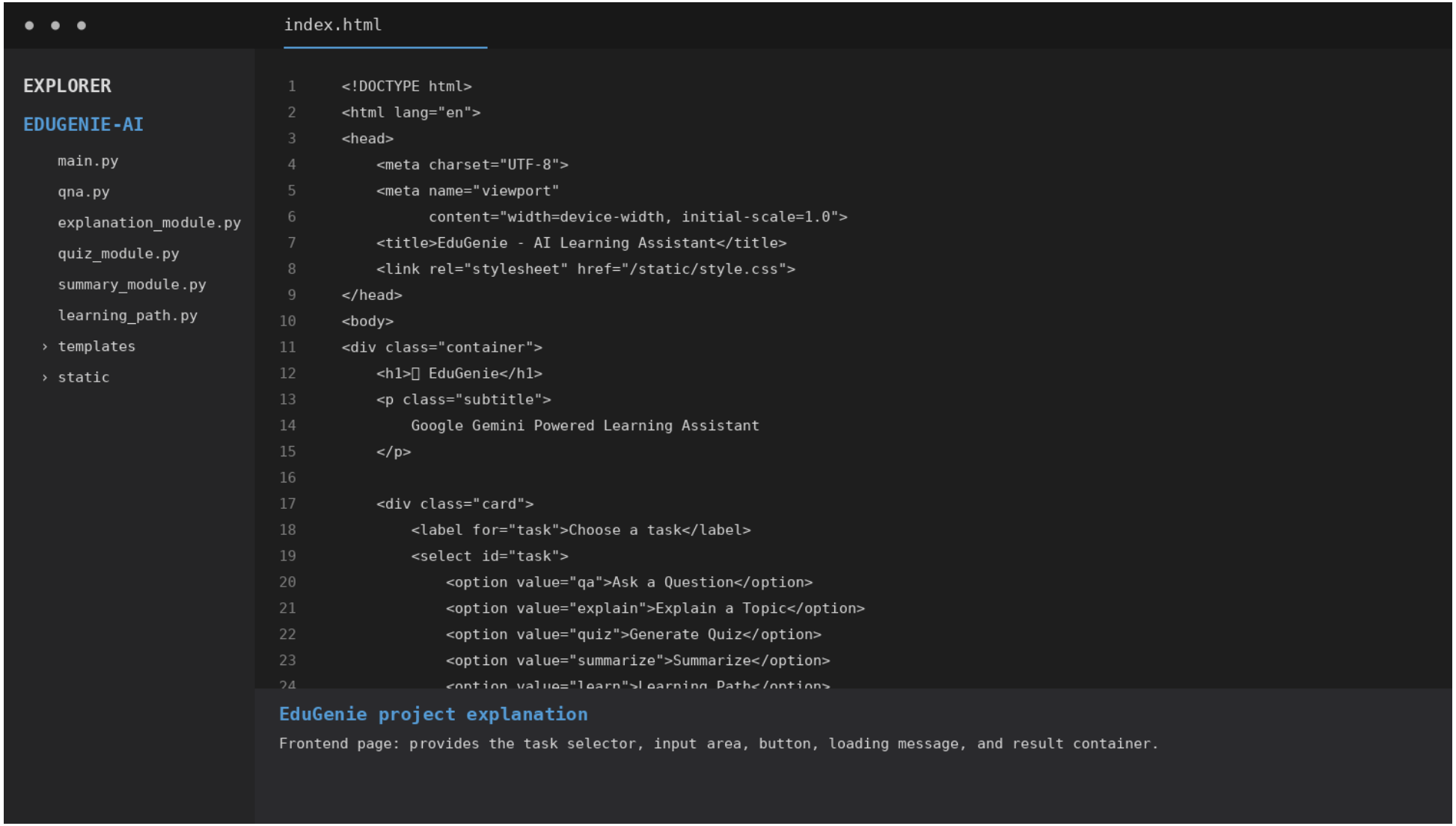


Fig. C7 – index.html

Explanation: Frontend page: provides the task selector, input area, button, loading message, and result container.

Coding Screenshot

style.css

EXPLORER

EDUGENIE-AI

- main.py
- qna.py
- explanation_module.py
- quiz_module.py
- summary_module.py
- learning_path.py
- templates
- static

```
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
}

body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #667eea, #764ba2);
    min-height: 100vh;
    padding: 40px 20px;
}

.container {
    max-width: 850px;
    margin: auto;
}

.card {
    background: white;
    padding: 30px;
    border-radius: 20px;
    box-shadow: 0 15px 35px rgba(0,0,0,0.2);
}
```

EduGenie project explanation

CSS styling: creates the responsive card layout, gradient background, input controls, and EduGenie button design.

Fig. C8 – style.css

Explanation: CSS styling: creates the responsive card layout, gradient background, input controls, and EduGenie button design.